Proof

We prove that the seven definitions are equivalent by showing the following cycle of implications:

[(1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (4) \Rightarrow (5) \Rightarrow (6) \Rightarrow (7) \Rightarrow (1)]

- **((1) \Rightarrow (2)):**
  By Definition 1, the graph is connected and has no cycles, so it is a forest. Since it is also connected, it must consist of exactly one connected component. Therefore, it is a connected component of a forest.

- **((2) \Rightarrow (3)):**
  From a standard graph theory result, a forest with (V) vertices, (E) edges, and (C) connected components satisfies [V - E = C.] Here the graph has only one connected component, so (C = 1). Hence, [E = V - 1.]

- **((3) \Rightarrow (4)):**
  A connected graph on (V) vertices needs at least (V - 1) edges. Since this graph has exactly (V - 1) edges, removing any edge would make it disconnected. So the graph is minimally connected.

- **((4) \Rightarrow (5)):**
  If the graph contained a cycle, then we could remove one edge from that cycle and the graph would still remain connected. This would contradict the assumption that the graph is minimally connected. So the graph must be acyclic. Also, any connected graph on (V) vertices must have at least (V - 1) edges, and here minimal connectedness gives exactly that number.

- **((5) \Rightarrow (6)):**
  If a graph is acyclic and has (V - 1) edges, then it must already be connected, so it is a tree. In such a graph, adding any new edge between two vertices must create a cycle. Therefore, the graph is maximally acyclic.

- **((6) \Rightarrow (7)):**
  If there were two vertices with no path between them, then adding an edge between them would not create a cycle, which would contradict maximal acyclicity. So there must be a path between every pair of vertices. On the other hand, if there were two distinct paths between the same pair of vertices, those paths would form a cycle. Hence, the path between any two vertices must be unique.

- **((7) \Rightarrow (1)):**
  Since there is a path between every pair of vertices, the graph is connected. Since that path is unique, there cannot be any cycle; otherwise, a cycle would provide more than one path between some pair of vertices. Thus, the graph is connected and acyclic.

Therefore, all seven definitions are equivalent.

In [1]:
```
import numpy as np
import pandas as pd

# Problem 2(a): Reconstruct Adjacency Matrices from CSC
def decode_csc(col_pointers, row_indices, nodes):
    """
    Decodes a Compressed Sparse Column (CSC) format into a dense adjacency matrix.
    Columns represent Destination nodes, Rows represent Source nodes.
    """
    n = len(nodes)
    matrix = np.zeros((n, n), dtype=int)

    for col in range(n):
        start_idx = col_pointers[col]
        end_idx = col_pointers[col + 1]
        for i in range(start_idx, end_idx):
            row = row_indices[i]
            matrix[row, col] = 1

    return pd.DataFrame(matrix, index=nodes, columns=nodes)

nodes = ['A', 'B', 'C', 'D', 'E']

print("--- (a) Adjacency Matrix for Graph 1 (Undirected) ---")
col_ptrs_1 = [0, 2, 5, 8, 11, 12]
row_idx_1 = [1, 2, 0, 2, 3, 0, 1, 3, 1, 2, 4, 3]
print(decode_csc(col_ptrs_1, row_idx_1, nodes))

print("\n--- (a) Adjacency Matrix for Graph 2 (Directed) ---")
col_ptrs_2 = [0, 0, 2, 4, 5, 7]
```

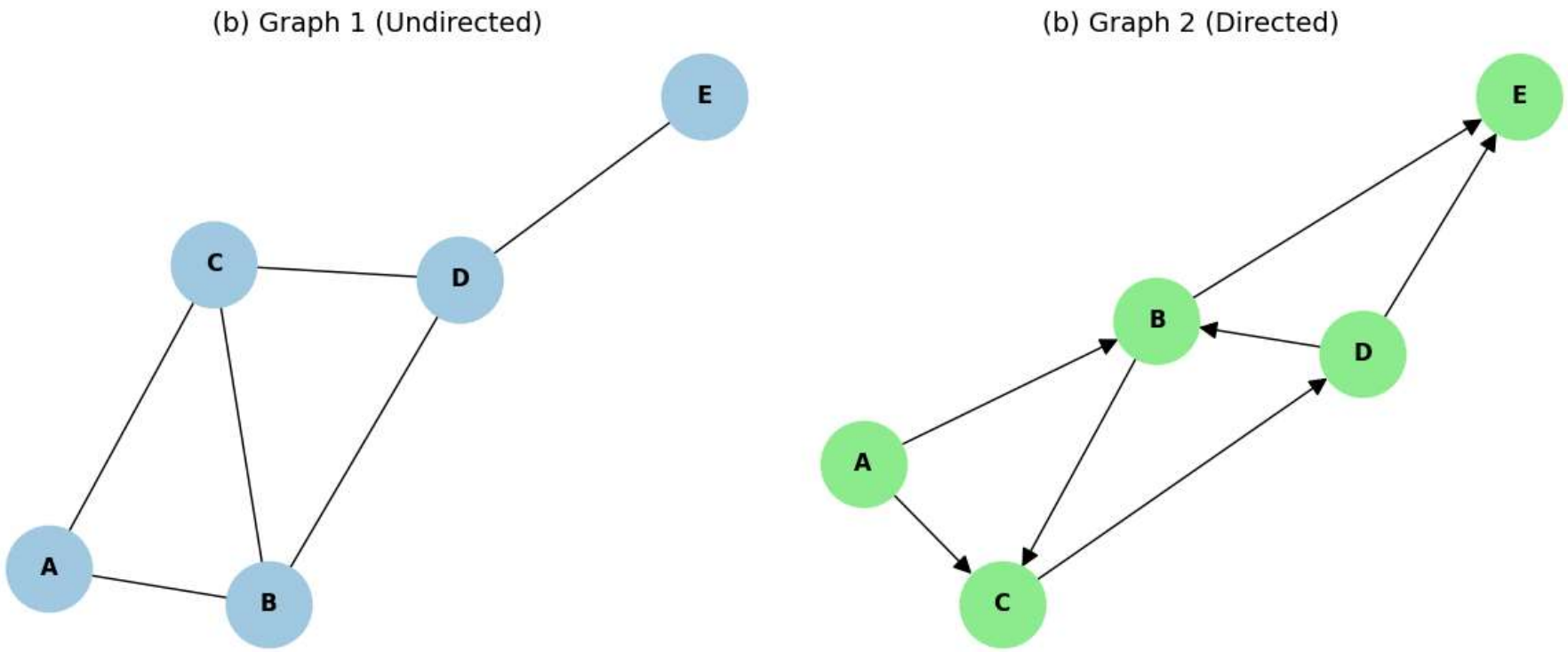
```
row_idx_2 = [0, 3, 0, 1, 2, 1, 3]
print(decode_csc(col_ptrs_2, row_idx_2, nodes))
```

```
--- (a) Adjacency Matrix for Graph 1 (Undirected) ---
   A  B  C  D  E
A  0  1  1  0  0
B  1  0  1  1  0
C  1  1  0  1  0
D  0  1  1  0  1
E  0  0  0  1  0

--- (a) Adjacency Matrix for Graph 2 (Directed) ---
   A  B  C  D  E
A  0  1  1  0  0
B  0  0  1  0  1
C  0  0  0  1  0
D  0  1  0  0  1
E  0  0  0  0  0
```

In [2]:

```
import networkx as nx
import matplotlib.pyplot as plt

# Problem 2(b): Graph Visualization
G1 = nx.Graph()
G2 = nx.DiGraph()

# Add edges reconstructed from CSC logic
G1.add_edges_from([('A', 'B'), ('A', 'C'), ('B', 'C'), ('B', 'D'), ('C', 'D'), ('D', 'E')])
G2.add_edges_from([('A', 'B'), ('D', 'B'), ('A', 'C'), ('B', 'C'), ('C', 'D'), ('B', 'E'), ('D', 'E')])

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
pos1 = nx.spring_layout(G1, seed=42)
pos2 = nx.spring_layout(G2, seed=42)

# Undirected Graph
nx.draw(G1, pos1, ax=axes[0], with_labels=True, node_color='#A0CBE2', node_size=2000, font_weight='bold')
axes[0].set_title("(b) Graph 1 (Undirected)", fontsize=14)

# Directed Graph
nx.draw(G2, pos2, ax=axes[1], with_labels=True, node_color='#90EE90', node_size=2000, font_weight='bold', arrows=True, arrowsi
axes[1].set_title("(b) Graph 2 (Directed)", fontsize=14)
```

```
plt.tight_layout()
plt.show()
```



In [3]:
```
# Problem 2(c): Find Unique Cycle in Directed Graph
cycles = list(nx.simple_cycles(G2))

print("--- (c) Unique Cycle in Directed Graph ---")
if cycles:
    for cycle in cycles:
        # Format the cycle list ['B', 'C', 'D'] into standard notation
        cycle_path = " -> ".join(cycle + [cycle[0]])
        print(f"The unique cycle detected by the algorithm is: {cycle_path}")
else:
    print("No cycles found.")
```

```
--- (c) Unique Cycle in Directed Graph ---
The unique cycle detected by the algorithm is: D -> B -> C -> D
```

In [ ]: